Sistema de Biblioteca em C

A ideia é fazermos um Sistema de Bibliotecas que vai sendo ampliado conforme vamos avançando no conteúdo da disciplina.

Você desenvolverá o mesmo sistema em seis versões. Cada versão acrescenta uma capacidade nova e reutiliza o que já funcionava.

O objetivo não é chegar rapidamente ao programa final, mas entender como cada conteúdo estudado resolve um problema real do sistema.

Conteúdos em destaque para cada versão:

1. Vetores
2. Strings
3. Funções
4. Structs
5. Ponteiros
6. Integração dos conteúdos

Como trabalhar neste projeto - Regra de ouro

Só avance quando a versão atual:

- Compilar sem erros;
- Executar corretamente;
- Passar pelos testes mínimos indicados.

Como ainda não utilizamos o git, salve uma cópia do programa antes de começar a próxima versão:

- biblioteca_v1.c
- biblioteca_v2.c
- biblioteca_v3.c
- biblioteca_v4.c
- biblioteca_v5.c
- biblioteca_v6.c

Roteiro do estudante para as 6 versões

Mapa das versões

Versão	Conteúdo novo	O sistema passa a conseguir...
1	Vetores e repetições	Controlar códigos e quantidades em estoque.
2	Strings	Mostrar uma descrição compreensível para cada livro.
3	Funções	Separar cadastro, listagem, busca e cálculos em operações.
4	Structs	Manter todos os dados de um livro na mesma entidade.
5	Ponteiros	Permitir que funções alterem livros e usuários originais.
6	Integração	Cadastrar, buscar, emprestar e devolver com regras completas.

Limites e regras do nosso protótipo

- No máximo **3 títulos de livros** e **3 usuários** distintos. OBS: isso significa que posso ter 3 títulos distintos, mas mais do que 3 exemplares de cada.
- Cada código de livro e/ou usuário deve ser um número inteiro, positivo e único.
- Cada livro reúne as seguintes informações: título, autor, ano e tema, em **uma única string**.
- Cada usuário reúne seus dados pessoais em **uma única string**.
- Cada usuário pode ficar com, no máximo, um livro emprestado por vez.
- Mensagens de erro não devem alterar os dados do sistema.

Sistema de Biblioteca - Versão 1

Nesta versão, o sistema deve conseguir:

- Cadastrar até 3 códigos numéricos de livros;
- Registrar a quantidade em estoque de cada livro;
- Listar todos os livros exibindo seus códigos e seus respectivos estoques;
- Calcular o total de exemplares da biblioteca;
- Localizar um livro pelo código;
- Informar se existe algum exemplar disponível.

Nesse momento, ainda não sabemos qual é o título, autor ou tema do livro. Cada livro será reconhecido somente por seu código numérico. Ou seja, todo o controle do estoque é feito via código e estoque.

Conteúdo em foco: vetores, for, while e estruturas de decisão.

Passo a passo

1. Crie uma constante "**MAX_LIVROS**" para representar a quantidade máxima de livros.
2. Crie um vetor "**codigos[MAX_LIVROS]**" para armazenar os códigos dos livros.
3. Crie um vetor "**estoques[MAX_LIVROS]**" para armazenar os estoques.
4. Crie uma variável "**qtdLivros = 0**" para controlar a quantidade de livros cadastrados.
5. Use um laço de repetição para ler três códigos e três quantidades de estoque.
6. Antes de aceitar um código, verifique se ele já foi cadastrado.
7. Não permita que o usuário informe um estoque negativo.
8. Use outro laço para listar o código e o estoque de cada livro.
9. Calcule e exiba o estoque total (utilizando a variável acumuladora" **qtdLivros"** inicializada com zero).
10. Solicite um código de livro para o usuário.
11. Realize a busca do código do livro fornecido pelo usuário (percorra o vetor procurando esse código).
12. Caso o código seja encontrado, mostre o estoque correspondente.
13. Informe se o livro está disponível, considerando:

Estoque maior que zero: disponível.

Estoque igual a zero: indisponível.

Relação entre os vetores

Os dados armazenados nos vetores códigos[i] e estoque[i] na mesma posição i pertencem ao mesmo livro.

Por exemplo:

codigos[1] = 205;

estoque[1] = 4;

Isso significa que o livro cujo código é 205 possui quatro exemplares em estoque.

Atenção aos índices

Como os vetores possuem três posições, os índices válidos são:

0, 1 e 2

Nunca devemos acessar a posição 3 de um vetor que possui tamanho 3.

Testes mínimos

1. Cadastre os seguintes valores:

Código	Estoque
101	2
205	0
310	5

O estoque total esperado é: **7**

2. Busque o código 205. Ele deve ser encontrado, mas deve aparecer como indisponível.
3. Busque o código 999. O sistema deve informar que o código não foi encontrado.
4. Tente cadastrar um código repetido. O código deve ser rejeitado.
5. Tente cadastrar um livro com estoque -1. O valor deve ser rejeitado.

A Versão 1 está concluída quando:

- Os vetores permanecem sincronizados;
- Todas as posições utilizadas são válidas;
- O programa localiza corretamente os códigos;
- O estoque total é calculado corretamente;
- Os quatro cenários de teste apresentam as mensagens esperadas.

Sistema de Biblioteca - Versão 2

Nesta versão, o sistema deve conseguir:

1. Além do código e do estoque, o sistema passa a exibir informações que uma pessoa consegue reconhecer:
    - Título;
    - Autor;
    - Ano;
    - Tema/categoria.
2. Formato combinado:
    - Todos os dados textuais do livro devem ser informados em uma única linha com no máximo 120 caracteres.
    - Para evitar a criação de muitas variáveis diferentes para título, autor, ano e categoria, apenas separe as informações utilizando ponto e vírgula, como por exemplo: "**O Hobbit; J. R. R. Tolkien; 1937; Fantasia**"

A busca continuará sendo realizada pelo código numérico do livro, mas agora o resultado também mostra a descrição do livro encontrado. Ou seja, temos a inclusão da descrição do livro. Como ainda não utilizamos matrizes nem structs, manteremos uma string para cada posição do catálogo.

Conteúdo em foco: strings como vetores de caracteres.

Passo a passo

1. Copie o código da Versão 1.
2. Crie uma constante "**TAM_TEXTO**" para representar a quantidade máxima de caracteres.
3. Acrescente as três strings que armazenarão as descrições:

    char descricao1[TAM_TEXTO];

    char descricao2[TAM_TEXTO];

    char descricao3[TAM_TEXTO];

4. Leia cada descrição utilizando **fgets**, pois essa função permite a leitura de espaços.
5. Depois de uma leitura numérica feita com **scanf**, trate o caractere \n que permaneceu no teclado antes de chamar **fgets**.
6. Durante a listagem, associe:

    Índice 0 -> descricao1

    Índice 1 -> descricao2

    Índice 2 -> descricao3

6. Mostre a descrição ao lado do código e do estoque.

7. Na busca por código, mostre também a descrição correspondente.

Observe que essa versão fica repetitiva devido a existência de três variáveis diferentes: descricao1, descricao2 e descricao3. Essa repetição é intencional. Na Versão 4, veremos como uma struct resolve esse problema e permite que cada livro tenha sua própria descrição.

Testes mínimos

1. Cadastre descrições que possuam espaços.
2. Cadastre descrições que possuam acentos.
3. Verifique se nenhuma descrição fica vazia por causa do \n.
4. Busque os três códigos e confira se cada um mostra a descrição correta.
5. Repita os testes de estoque realizados na Versão 1.

A Versão 2 está concluída quando:

Cada código aparece sempre com:

- Sua própria descrição;
- Seu próprio estoque;
- A situação correta de disponibilidade.

Essa associação deve permanecer correta tanto na listagem quanto na busca.

Sistema de Biblioteca - Versão 3

Nesta versão, o sistema deixará de ser um único bloco de comandos. As diferentes atividades do programa serão separadas em funções, como:

- cadastro de títulos;
- listagem do acervo;
- busca por código;
- cálculo do total de exemplares;
- verificação da disponibilidade.

A separação em funções torna o programa mais fácil de ler, testar, corrigir, reutilizar e modificar.

Conteúdos em foco: funções, parâmetros, valores de retorno e escopo de variáveis.

Antes de começar

Utilize como ponto de partida o programa concluído na versão 2. Portanto, o sistema continuará utilizando:

int codigos[MAX_TITULOS];

int estoques[MAX_TITULOS];

char descricao1[TAM_TEXTO];

char descricao2[TAM_TEXTO];

char descricao3[TAM_TEXTO];

Nesta versão, as três descrições continuarão separadas. Essa limitação será resolvida na versão 4, quando cada título será organizado em uma struct.

Protótipos das funções

Os protótipos informam ao compilador:

- o nome da função;
- o tipo de valor que ela devolve;
- os dados que ela precisa receber.

Escreva os protótipos depois das constantes e antes da função main. As implementações completas das funções poderão ser colocadas depois da main.

```
int cadastrarTitulo(
    int codigos[],
    int estoques[],
    char descricao1[],
    char descricao2[],
    char descricao3[],
    int qtdTitulos
);

void listarTitulos(
    int codigos[],
    int estoques[],
    char descricao1[],
    char descricao2[],
    char descricao3[],
    int qtdTitulos
);

int buscarTitulo(
    int codigos[],
    int qtdTitulos,
    int codigoBuscado
);

int calcularQtdExemplares(
    int estoques[],
    int qtdTitulos
);
int disponibilidadeTitulo(
    int qtdEstoque
);
```

Operações do sistema

1. Função para cadastrar um título

```
int cadastrarTitulo(
  int codigos[],
  int estoques[],
  char descricao1[],
  char descricao2[],
  char descricao3[],
  int qtdTitulos
);
```

Essa função deve:

1. Solicitar o código do novo título.
2. Impedir o cadastro de códigos negativos.
3. Verificar se o código já foi cadastrado.
4. Solicitar a quantidade de exemplares em estoque.
5. Impedir o cadastro de estoques negativos.
6. Solicitar a descrição do título.
7. Armazenar o código, o estoque e a descrição.
8. Incrementar e devolver a nova quantidade de títulos cadastrados.

A chamada na main deverá guardar o valor retornado:

```
qtdTitulos = cadastrarTitulo(
  codigos,
  estoques,
  descricao1,
  descricao2,
  descricao3,
  qtdTitulos
);
```

Isso é necessário porque qtdTitulos é uma variável inteira passada por valor. O incremento realizado dentro da função não modifica diretamente a variável existente na main.

Os vetores e as strings, por sua vez, podem ter seus elementos alterados pela função.

2. Função para listar todos os títulos

```
void listarTitulos(
  int codigos[],
  int estoques[],
  char descricao1[],
  char descricao2[],
  char descricao3[],
  int qtdTitulos
);
```

Essa função deve percorrer todos os títulos cadastrados e mostrar:

- o código;
- a descrição;
- a quantidade de exemplares em estoque.

A função não precisa devolver nenhum valor. Por isso, seu tipo de retorno é void.

Exemplo de chamada:

```
listarTitulos(
  codigos,
  estoques,
  descricao1,
  descricao2,
  descricao3,
  qtdTitulos
);
```

O laço responsável pela listagem deve estar dentro da função, e não na main.

3. Função para buscar um título

```
int buscarTitulo(
  int codigos[],
  int qtdTitulos,
  int codigoBuscado
);
```

Essa função deve percorrer o vetor codigos procurando o código fornecido.

Ela deve devolver:

- o índice do título, quando o código for encontrado;
- -1, quando o código não for encontrado.

Exemplo de chamada:

```
int posBusca;


posBusca = buscarTitulo(
  codigos,
  qtdTitulos,
  codigoBuscado
);
```

Depois da chamada, a main deverá analisar o resultado:

```
if (posBusca == -1) {
  printf("Titulo nao encontrado.\n");
}
else {
  printf("Titulo encontrado na posicao %d.\n", posBusca);
}
```

A função de busca não precisa imprimir se o título foi encontrado. Sua responsabilidade é somente realizar a busca e devolver o resultado.

Dessa maneira, ela poderá ser reutilizada posteriormente pelo cadastro, empréstimo, devolução e outras operações.

4. Função para calcular o total de exemplares

```
int calcularQtdExemplares(
  int estoques[],
  int qtdTitulos
);
```

Essa função deve:

1. Criar uma variável acumuladora inicializada com zero.
2. Percorrer os estoques dos títulos cadastrados.
3. Somar cada estoque ao acumulador.
4. Devolver a quantidade total de exemplares.

Exemplo de chamada:

```
qtdExemplares = calcularQtdExemplares(
  estoques,
  qtdTitulos
);
```

Atenção: o acumulador deve obrigatoriamente ser inicializado:

```
int qtdExemplares = 0;
```

5. Função para verificar a disponibilidade

```
int disponibilidadeTitulo(
  int qtdEstoque
);
```

Essa função deve receber a quantidade em estoque de um único título e devolver:

- 1, quando o estoque for maior que zero;
- 0, quando o estoque for igual a zero.

Exemplo:

```
disponivel = disponibilidadeTitulo(
  estoques[posBusca]
);
```

O programa só poderá acessar estoques[posBusca] depois de verificar se posBusca é diferente de -1.

Depois da chamada, a main poderá apresentar a mensagem apropriada:

```
if (disponivel == 1) {
  printf("Titulo disponivel.\n");
}
else {
  printf("Titulo temporariamente indisponivel.\n");
}
```

Passo a passo para desenvolver a versão 3

1. Copie o programa concluído na versão 2.
2. Identifique os trechos responsáveis pelo cadastro, pela listagem, pela busca, pelo cálculo do estoque total e pela disponibilidade.
3. Escreva os protótipos das funções antes da main.

4. Transforme o cadastro em uma função.
5. Substitua o cadastro existente na main pela chamada da nova função.
6. Compile e teste o programa.
7. Transforme a listagem em uma função.
8. Compile e teste novamente.
9. Transforme a busca em uma função que devolva o índice encontrado ou -1.
10. Transforme o cálculo do total de exemplares em uma função.
11. Transforme a verificação da disponibilidade em uma função.
12. Faça cada função receber somente os dados necessários para realizar sua tarefa.
13. Mantenha a main responsável por organizar a sequência das operações.
14. Teste o programa completo.

Não transforme todas as partes ao mesmo tempo. Crie e teste uma função antes de iniciar a seguinte. Assim, caso apareça um erro, será mais fácil identificar qual alteração o provocou.

Como deve ficar a função main

Ao final, a função main deve funcionar como um roteiro das operações realizadas pelo sistema:

1. cadastrar os títulos;
2. listar os títulos;
3. solicitar um código para busca;
4. buscar o título;
5. verificar sua disponibilidade;
6. calcular e mostrar o total de exemplares.

Os detalhes de cada operação devem estar nas respectivas funções.

Testes mínimos

Testes do cadastro

- Cadastre um código válido.
- Tente cadastrar um código negativo.
- Tente cadastrar um código repetido.
- Cadastre um título com estoque igual a zero.
- Tente cadastrar um estoque negativo.
- Verifique se a descrição foi armazenada no título correto.

Testes da busca

- Busque o primeiro título cadastrado.
- Busque o último título cadastrado.
- Busque um código inexistente.
- Confirme que a função devolve -1 quando o código não existe.

Testes do estoque

- Calcule a soma dos estoques 0, 0 e 0. O resultado esperado é 0.
- Calcule a soma de três estoques diferentes de zero.
- Verifique a disponibilidade de um título com estoque igual a zero.
- Verifique a disponibilidade de um título com estoque igual a um.

Teste da listagem

- Faça uma listagem completa sem repetir o laço de listagem na função main.
- Confirme que cada código aparece acompanhado do estoque e da descrição correspondentes.

A versão 3 estará concluída quando

- o cadastro estiver em uma função;
- a listagem estiver em uma função;
- a busca devolver um índice ou -1;
- o cálculo do total de exemplares estiver em uma função;
- a disponibilidade for informada por uma função;
- cada função possuir uma responsabilidade clara;
- cada função receber apenas os dados necessários;
- a função main puder ser lida como um roteiro de operações;
- não existirem cópias desnecessárias do mesmo código;
- o programa apresentar o mesmo comportamento da versão 2, mas com melhor organização.

Sistema de Biblioteca - Versão 4

Nesta versão, cada livro vira uma entidade organizada: código, descrição e estoque de livros deixam de ficar espalhados em diferentes vetores e variáveis. Cada posição do vetor passa a representar um livro completo. Também começamos a organizar os usuários para preparar as operações de empréstimo e devolução.

Conteúdo em foco: structs e vetores de structs.

Representações sugeridas

1. Representação de um livro

```
typedef struct {
  int codigo;
  char descricao[TAM_TEXTO];
  int estoque;
} Livro;
```

- Agora podemos criar um vetor de livros:
  Livro livros[MAX_LIVROS];

Cada elemento do vetor possui todos os dados de um livro.

Exemplo:

livros[0].codigo = 101;

livros[0].estoque = 2;

A descrição pode ser acessada com:

livros[0].descricao

2. Representação de um usuário

```
typedef struct {
  int codigo;
  char dados[TAM_TEXTO];
  int codigoLivroEmprestado;
} Usuario;
```

- Podemos criar um vetor de usuários:
  Usuario usuarios[MAX_USUARIOS];

Como nenhum código válido de livro deve possuir o valor -1, para indicar que o usuário não está com nenhum livro emprestado usaremos: “**codigoLivroEmprestado = -1;**”

Passo a passo

1. Crie o tipo Livro.
2. Crie o tipo Usuario.
3. Substitua os vetores paralelos por:

Livro livros[MAX_LIVROS];

4. Atualize os acessos aos dados.
    - Antes:

      codigos[i]

      estoques[i]
    - Depois:

      livros[i].codigo

      livros[i].descricao

      livros[i].estoque
5. Adapte a função de listagem para receber um vetor de livros.
6. Adapte a busca para trabalhar com o campo codigo da struct.
7. Adapte o cálculo do estoque total.
8. Cadastre até três usuários.
9. Inicialize o campo codigoLivroEmprestado de cada usuário com -1.

Testes mínimos

1. Liste três livros e confira se nenhum campo foi trocado.
2. Busque um livro e mostre seu código, sua descrição e seu estoque.
3. Liste três usuários recém-cadastrados.
4. Confirme que todos os novos usuários possuem codigoLivroEmprestado igual a -1.
5. Confirme que os limites de três livros e três usuários continuam sendo respeitados.

A Versão 4 está concluída quando:

- Não existem mais vetores paralelos para representar os livros;
- Cada elemento de livros contém todos os dados da mesma entidade;
- Os usuários também estão representados de forma organizada;
- As funções conseguem trabalhar com vetores de structs.

Sistema de Biblioteca - Versão 5

Nesta versão, uma função passa a conseguir alterar o objeto original recebido, seja livros ou usuários. Isso permite:

- Modificar o estoque de um livro;
- Registrar qual livro está com um usuário;
- Liberar o usuário após uma devolução;
- Atualizar os dados sem depender de variáveis globais.

Conteúdo em foco: ponteiros e passagem por endereço.

Operações sugeridas

1. Alteração do estoque:

```
void alterarEstoque(
  Livro *livro,
  int valor
);
```

2. Registro de empréstimo:

```
void registrarEmprestimo(
  Usuario *usuario,
  int codigoLivro
);
```

3. Devolução de livro:

```
void encerrarEmprestimo(
  Usuario *usuario
);
```

Acesso aos campos por ponteiro

Dentro da função, utilizamos o operador ->:

```
livro->estoque = valor;
```

Para alterar a situação do usuário:

```
usuario->codigoLivroEmprestado = codigoLivro;
```

Para encerrar o empréstimo:

usuario->codigoLivroEmprestado = -1;

Enviando o endereço

Ao chamar uma função que recebe um ponteiro, devemos enviar o endereço do elemento:

alterarEstoque(&livros[indice], 1);

Para um usuário:

```
registrarEmprestimo(
  &usuarios[indiceUsuario],
  codigoLivro
);
```

Passo a passo

1. Copie o programa da Versão 4.
2. Mantenha as funções que apenas consultam os dados.
3. Crie pequenas funções que recebam Livro * ou Usuario *.
4. Dentro das funções, altere os campos utilizando ->.
5. Na função main, chame as funções passando o endereço do elemento desejado.
6. Após cada chamada, imprima o objeto na função main.
7. Confirme que a alteração feita dentro da função continuou existindo depois que a função terminou.

Testes mínimos

- Altere o estoque de um livro de 2 para 1.
- Depois da função terminar, confirme que o valor continua sendo 1.
- Registre o código 101 em um usuário que possuía codigoLivroEmprestado igual a -1.
- Encerre o empréstimo desse usuário.
- Confirme que o campo voltou para -1.
- Faça duas alterações seguidas e verifique se elas se acumulam.

Versão 5 concluída quando...

- As alterações feitas dentro das funções continuam visíveis depois que a função termina;
- As funções recebem o endereço correto dos objetos;
- O acesso aos campos por ponteiros é feito com ->;
- Nenhum dado depende de variável global.

Sistema de Biblioteca - Versão 6

Nesta versão, o sistema passa a operar um pequeno acervo do início ao fim. É possível:

- Cadastrar livros;
- Cadastrar usuários;
- Listar livros;
- Buscar registros;
- Emprestar um exemplar;
- Devolver um exemplar;
- Atualizar o estoque;
- Manter a situação do usuário consistente.

Conteúdo em foco: integração de vetores, strings, funções, structs e ponteiros.

Ordem de integração

Integre uma operação de cada vez. Execute os testes de cada etapa antes de adicionar a operação seguinte.

Etapa	Implementação	Verificação imediata
1	Crie o laço do menu e a opção 0.	Uma opção inválida repete o menu e 0 encerra o programa.
2	Integre o cadastro de livros e usuários.	Os limites e códigos repetidos são tratados.
3	Integre a listagem e as duas buscas.	Primeiro, último e inexistente funcionam corretamente.
4	Implemente o empréstimo com as validações.	O sucesso modifica livro e usuário; a falha não modifica nada.
5	Implemente a devolução com as validações.	O estoque e o usuário retornam ao estado correto.
6	Revise mensagens, nomes e organização.	Outra pessoa consegue operar o sistema sem explicação oral.

Regras obrigatórias para cadastro:

- Não cadastrar mais de três livros.
- Não cadastrar mais de três usuários.
- Não aceitar código de livro repetido.
- Não aceitar código de usuário repetido.
- Não aceitar código negativo ou igual a zero.
- Não aceitar estoque negativo.

Regras obrigatórias em caso de falhas

Se qualquer regra falhar:

- O sistema deve mostrar uma mensagem;
- Nenhum dado deve ser alterado;
- O programa deve voltar ao menu.

Retorno das operações:

As funções **emprestarLivro e devolverLivro** podem retornar:

1 - operação realizada com sucesso

0 - operação não realizada

Exemplo:

```
if (emprestarLivro(
      &livros[indiceLivro],
      &usuarios[indiceUsuario]
   )) {
   printf("Emprestimo realizado.\n");
} else {
   printf("Nao foi possivel realizar o emprestimo.\n");
}
```

O programa deve apresentar:

1. **Menu mínimo com as seguintes opções:**

1 - Cadastrar livro

2 - Cadastrar usuário

3 - Listar livros

4 - Buscar livro

5 - Emprestar livro

6 - Devolver livro

0 – Sair

2. Funções centrais:

- **Buscar livro:**

```
int buscarLivro(
  Livro livros[],
  int qtd,
  int codigo
);
```

- **Buscar usuário:**

```
int buscarUsuario(
  Usuario usuarios[],
  int qtd,
  int codigo
);
```

- **Emprestar livro:**

```
int emprestarLivro(
  Livro *livro,
  Usuario *usuario
);
```

Regras de Empréstimo:

O empréstimo só pode acontecer quando:

- O livro existe;
- O usuário existe;
- O estoque do livro é maior que zero;
- O usuário não possui outro livro emprestado.

Quando o empréstimo for realizado:

Estoque do livro diminui 1.

Usuário recebe o código do livro.

Um empréstimo envolve duas mudanças que devem acontecer juntas:

Livro: estoque diminui 1.

Usuário: recebe o código do livro.

Uma devolução realiza o processo inverso:

Livro: estoque aumenta 1.

Usuário: código do livro volta para -1.

Não pode existir uma situação em que apenas um dos objetos seja atualizado.

- **Devolver livro:**

```
int devolverLivro(
  Livro *livro,
  Usuario *usuario
);
```

Regras de devolução:

A devolução só pode acontecer quando:

- O usuário existe;
- O usuário possui um livro emprestado;
- O livro existe;
- O código do livro corresponde ao empréstimo registrado no usuário.

Quando a devolução for realizada:

Estoque do livro aumenta 1.

Código do livro no usuário volta para -1.

Roteiro de teste final

Teste 1 - Cadastro inicial

Cadastre:

Livro:

Código: 101

Estoque: 2

Usuário:

Código: 1

Resultado esperado:

- Os dois cadastros são aceitos;
- O usuário inicia com codigoLivroEmprestado = -1.

Teste 2 - Empréstimo válido

Empreste o livro 101 ao usuário 1.

Resultado esperado:

- Estoque do livro 101: 1
- Livro registrado no usuário 1: 101

Teste 3 - Usuário já possui um livro

Tente realizar outro empréstimo para o usuário 1.

Resultado esperado:

- A operação é recusada;
- O estoque não é alterado;
- O código armazenado no usuário continua sendo 101.

Teste 4 - Devolução válida

Devolva o livro 101 pelo usuário 1.

Resultado esperado:

- Estoque do livro 101: 2
- Livro registrado no usuário 1: -1

Teste 5 - Devolução repetida

Tente devolver novamente o livro pelo mesmo usuário.

Resultado esperado:

- A operação é recusada;
- O estoque continua sendo 2;
- O usuário continua com o valor -1.

Teste 6 - Livro sem estoque

Cadastre um livro com estoque zero e tente emprestá-lo.

Resultado esperado:

- O empréstimo é recusado;
- O estoque continua sendo zero;
- O usuário continua sem empréstimo.

Teste 7 - Códigos inexistentes

Use códigos inexistentes nas seguintes operações:

- Busca;
- Empréstimo;
- Devolução.

Resultado esperado:

- O sistema mostra mensagens claras;
- Nenhum dado é alterado;
- O menu continua funcionando.

A Versão 6 está concluída quando:

- Todos os testes finais são aprovados;
- O menu continua funcionando depois de operações inválidas;
- As regras preservam a relação entre livro, estoque e usuário;
- O sistema não altera parcialmente os dados;
- O programa continua limitado a três livros e três usuários.

Checklist de entrega

Código e funcionamento

- O programa compila sem erros.
- O programa utiliza constantes para os limites.
- Nenhuma posição inválida dos vetores é acessada.
- Códigos repetidos são tratados.
- Códigos inexistentes são tratados.
- Strings com espaços são lidas corretamente.
- Cada função possui uma responsabilidade clara.
- As alterações nas structs utilizam ponteiros quando necessário.
- O empréstimo modifica o livro e o usuário.
- A devolução modifica o livro e o usuário.
- Operações inválidas não alteram os dados.

Organização e comunicação

- Existem seis arquivos, um para cada versão.
- As variáveis possuem nomes compreensíveis.
- As funções possuem nomes compreensíveis.
- A indentação é consistente.
- Os comentários explicam decisões importantes.
- Os comentários não repetem comandos óbvios.
- As mensagens orientam o usuário do programa.
- Consegue explicar o que mudou em cada versão.
- Consegue demonstrar as operações.

Se você tivesse que continuar o desenvolvimento do projeto, qual seria a **Versão 7**?

- Qual novo problema essa versão resolveria;
- Qual funcionalidade seria acrescentada;
- Qual novo conteúdo da linguagem C seria necessário;
- Como essa nova versão aproveitaria o programa já desenvolvido.